Using Amazon S3 to Handle Large Items in DynamoDB

To Begin with the Lab

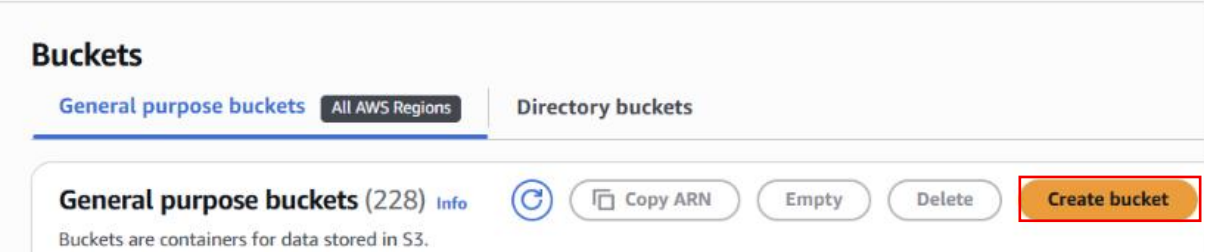
Summary of the Lab

This lecture explains how to handle large DynamoDB item attributes by storing them in Amazon S3 instead of using Gzip compression. It covers creating an S3 bucket, assigning IAM permissions, uploading content to S3, saving the S3 URL in DynamoDB, and retrieving the content back from S3. This approach is ideal for storing large files such as images, audio, and videos.

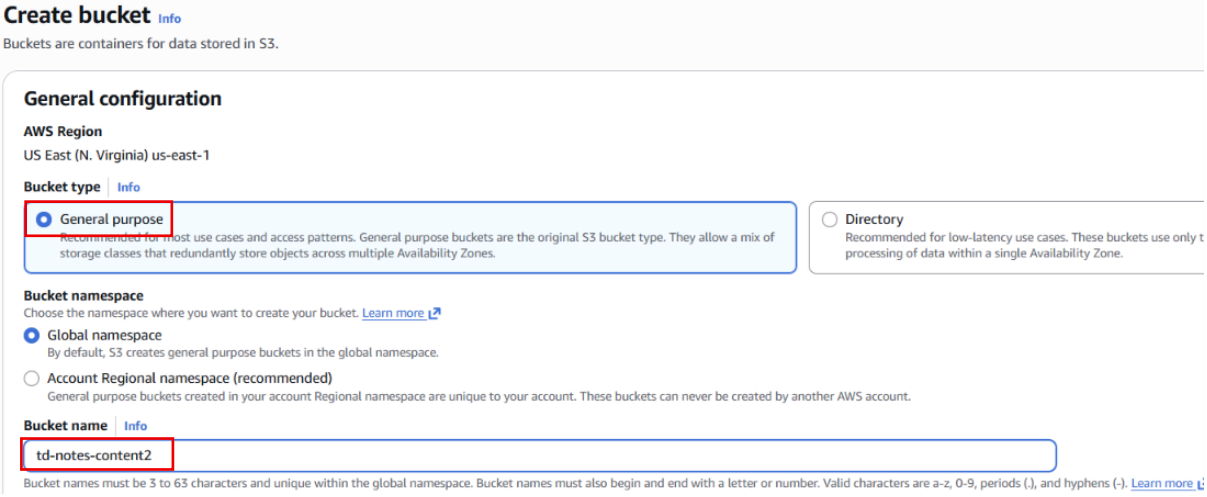
Lab steps

Step 1: Create an S3 Bucket

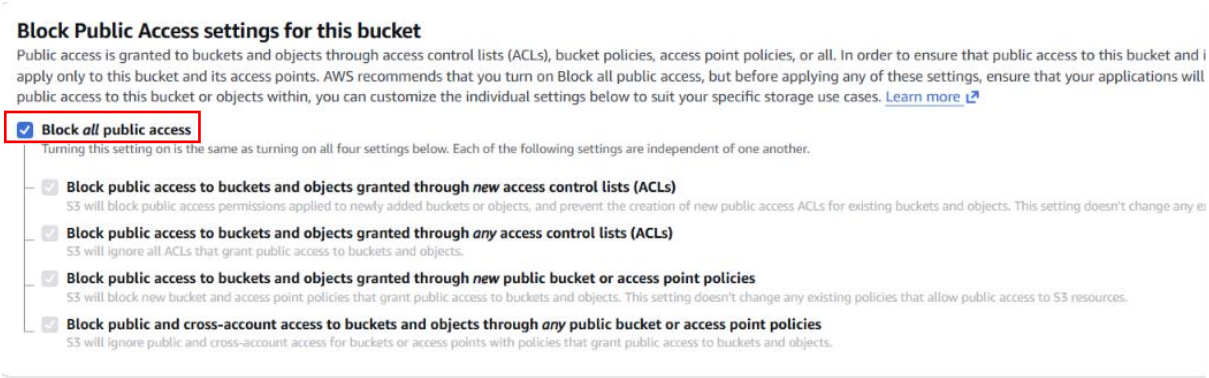
- Go to Amazon S3 in AWS Account
- Following the steps to Create bucket

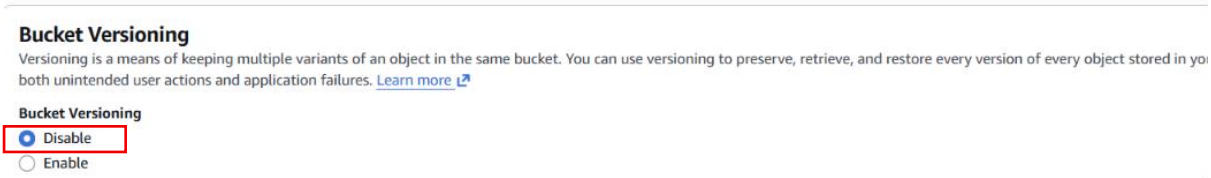


- Select General Purpose Bucket time
- Provide the name to identify the bucket

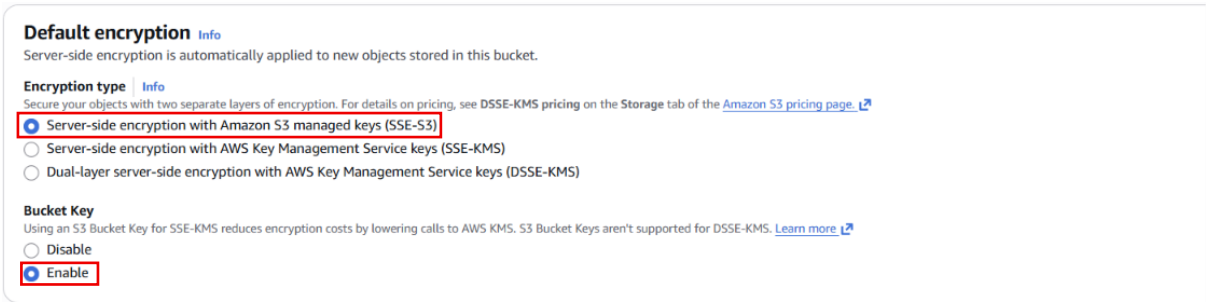


- Choose the Block all public access
- Disable the Bucket versioning

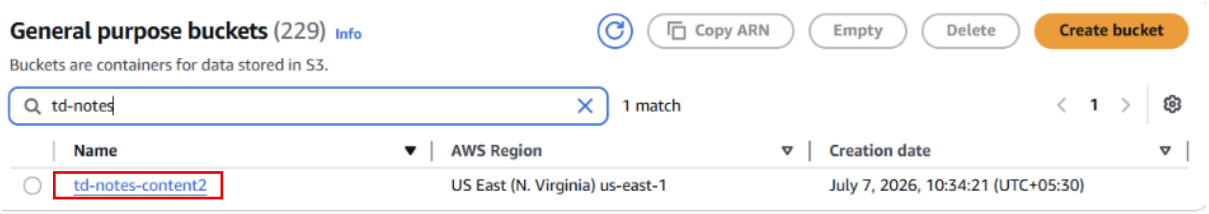




- Go to Default encryption section
- Choose Encryption type**, Server-side encryption with AmazonS3 managed keys (SSE-S3)**
- Enable the **Bucket Key**
- Click on the **Create Bucket**

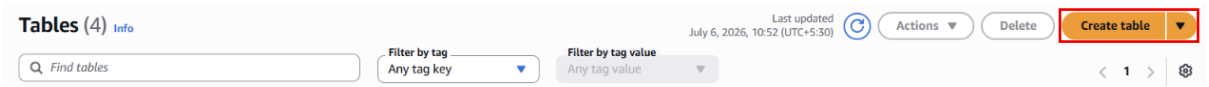


- Bucket is Created

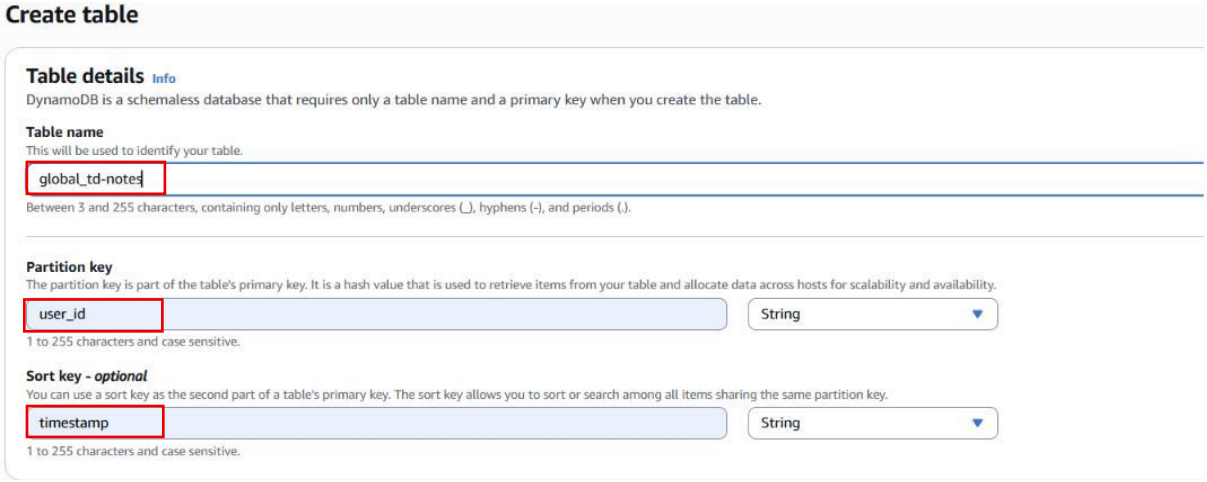


Step 2: Create a DynamoDB Table

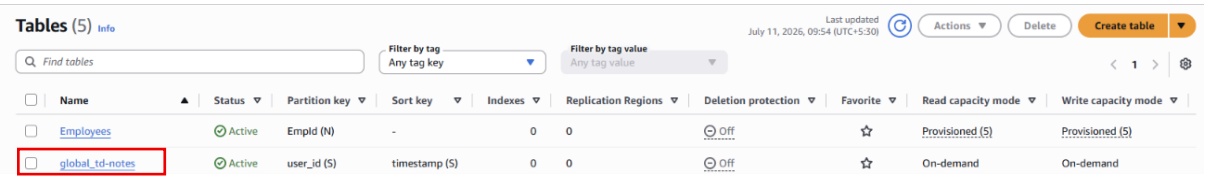
- Go to DynamoDB in AWS Account
- Under DynamoDB go to Tables
- Create a table



- Provide the Table Name **global_td-notes** to identify the Table
- Provide the **partition key** for scalability and availability
- Provide a **sort key** for allows you to sort or search among all items
- Click on **Create Table**

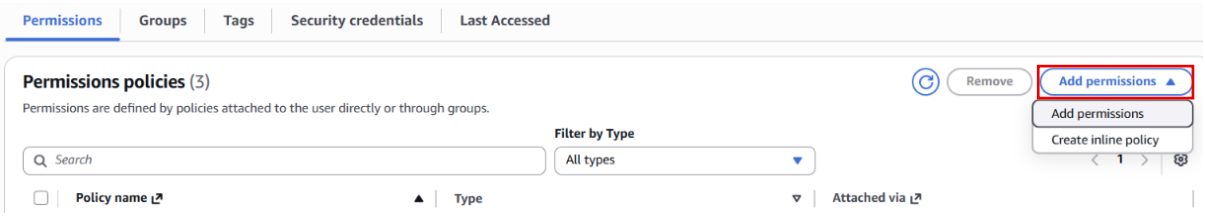


- Table is Created

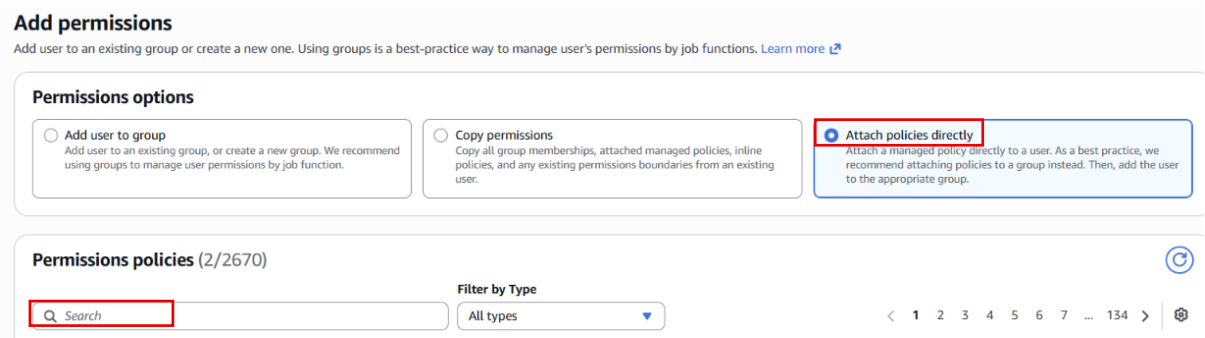


Step 3: Create IAM Role

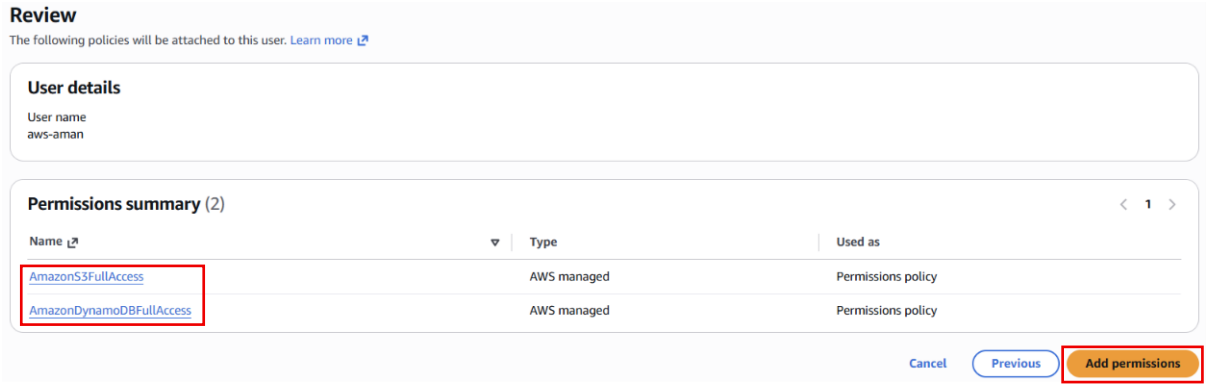
- Go to the IAM in AWS Account
- Then go to **IAM** User, select own **user**
- Click on the **Add Permission**



- Choose the **Attach policy directly** in Add permission
- Go to the search bar select two policies
  1.AmazonS3FullAccess
  2.AmazonDynamoDBFullAccess

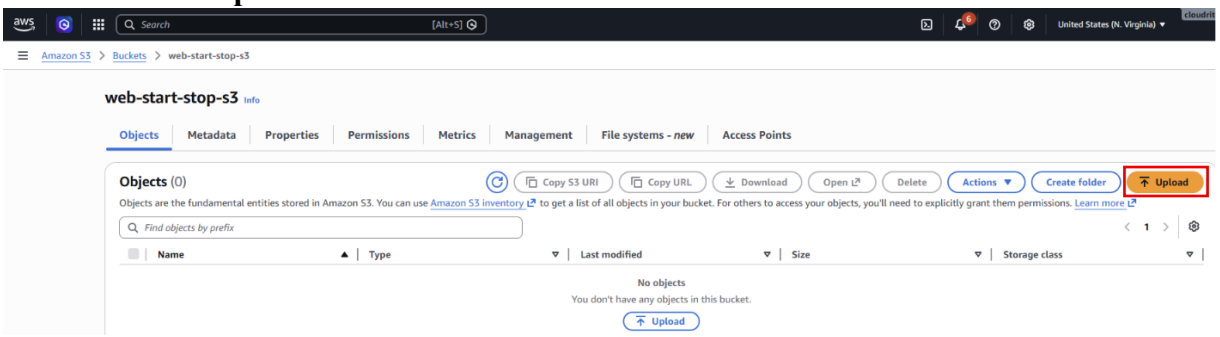


- These are the two policies that you are select
- Click on the **Add permission**

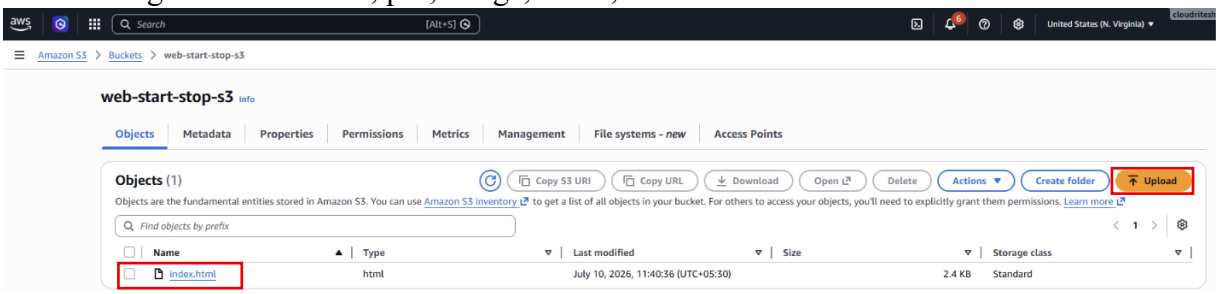


Step 4: Upload a File to S3

- Click on the **upload**

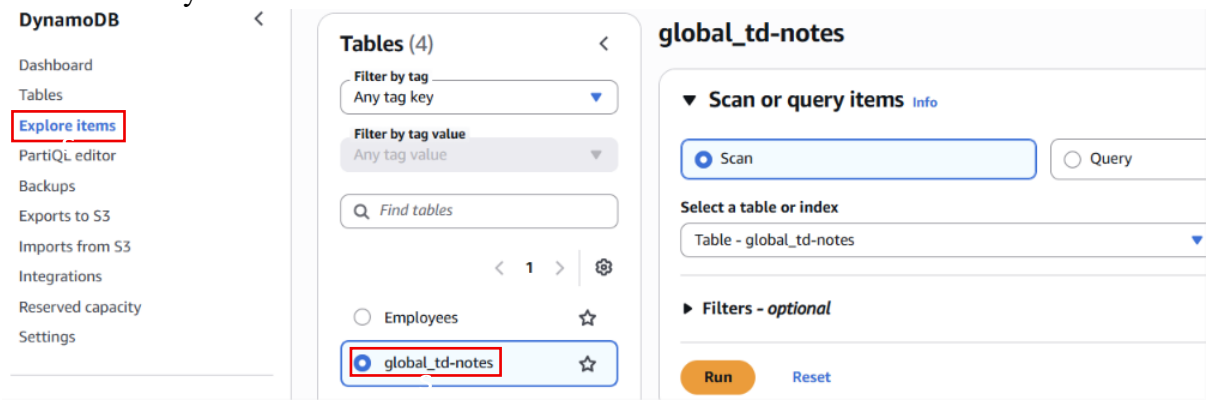


- Click on the **Add file**
- Add large file like media, pdf, image, video, or text file



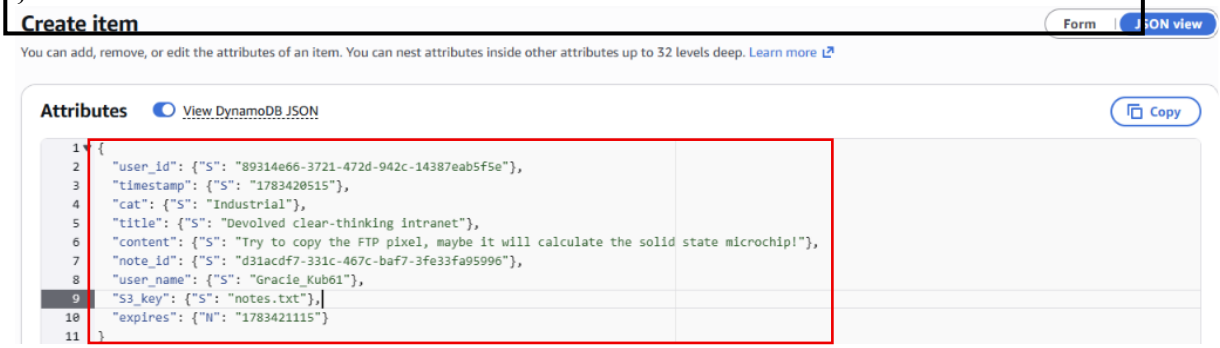
Step 5: Create an Item in DynamoDB

- Go to the DynamoDB in AWS Account
- Under DynamoDB, click on the **Explore items**
- Select the your Table

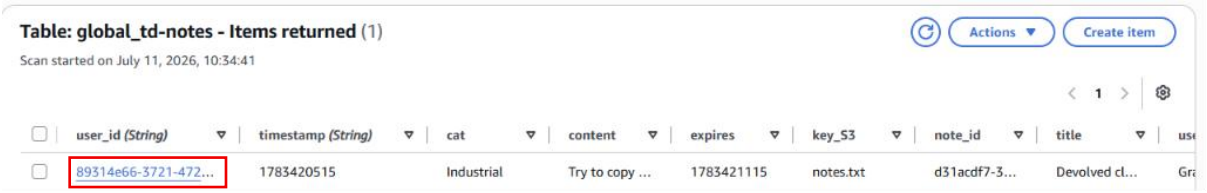


- Click on the **create item**
- Paste this **json code**
- Copy the S3_key


```
{
 "user_id": {"S": "89314e66-3721-472d-942c-14387eab5f5e"},
 "timestamp": {"S": "1783420515"},
 "cat": {"S": "Industrial"},
 "title": {"S": "Devolved clear-thinking intranet"},
 "content": {"S": "Try to copy the FTP pixel, maybe it will calculate the solid state microchip!"},
 "note_id": {"S": "d31acdf7-331c-467c-baf7-3fe33fa95996"},
 "user_name": {"S": "Gracie_Kub61"},
 "S3_key": {"S": "notes.txt"},
 "expires": {"N": "1783421115"}
}
```

- Item is created



Step 6: Retrieve Metadata from DynamoDB

- Search the code on the AI Application
- Implement code for S3 + DynamoDB Large Item Storage using AWS SDK for JavaScript v3
- Create a File **large-item-S3.js**, open with Vs code
- Paste this code in large-item-S3 file
- Download the file in Resources

```
large-items-S3.js > ...
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
const {
  DynamoDBDocumentClient,
  PutCommand,
  GetCommand,
} = require("@aws-sdk/lib-dynamodb");
const {
  S3Client,
  PutObjectCommand,
  GetObjectCommand,
} = require("@aws-sdk/client-s3");
const faker = require("faker");
const moment = require("moment");

const REGION = "us-east-1";
const TABLE_NAME = "global_td-notes";
const BUCKET_NAME = "td-notes-content2";

// DynamoDB Client
const dynamoClient = new DynamoDBClient({
  region: REGION,
});

const docClient = DynamoDBDocumentClient.from(dynamoClient);

// S3 Client
const s3Client = new S3Client({
  region: REGION,
});

(async () => {
  try {
    const item = generateNotesItemS3();

    console.log("Original Item");
    console.log(item);

    await putNotesItemS3(item);

    console.log("\nStored Item");

    const data = await getNotesItemS3({
      user_id: item.user_id,
      timestamp: item.timestamp,
    });

    console.log("\nRetrieved Item");
    console.log(data.Item);

  } catch (err) {
    console.error("ERROR:");
```

- Go to the Terminal
- If AWS SDK is not Install, installed the AWS SDK
- Implement this command Run in the terminal, **node Large-Item-S3.js**
- Item is stored successfully
- Amazon S3 is use to store large scale item like Media object

```
PS C:\Users\rrath\OneDrive\Documents\Sandbox> node large-items-S3.js
Original Item:
{
  user_id: '89314e66-3721-472d-942c-14387eab5f5e',
  timestamp: '1783420515',
  cat: 'Industrial',
  title: 'Devolved clear-thinking intranet',
  content: 'Try to copy the FTP pixel, maybe it will calculate the solid state microchip!',
  note_id: 'd31acdf7-331c-467c-baf7-3fe33fa95996',
  user_name: 'Gracie_Kub61',
  expires: 1783421115
}

Item stored successfully.

Retrieved Item:
{
  user_id: '89314e66-3721-472d-942c-14387eab5f5e',
  expires: 1783421115,
  cat: 'Industrial',
  note_id: 'd31acdf7-331c-467c-baf7-3fe33fa95996',
  user_name: 'Gracie_Kub61',
  title: 'Devolved clear-thinking intranet',
  timestamp: '1783420515',
  content: 'Try to copy the FTP pixel, maybe it will calculate the solid state microchip!'
}
PS C:\Users\rrath\OneDrive\Documents\Sandbox>
```